

Aula 17: Neural ODEs e PINNs

Neural Differential Equations e Physics-Informed Networks

Disciplina de Aprendizagem Profunda em Tempo Contínuo

Objetivos da aula

- Relembrar EDOs e implementar o método de Euler em NumPy.
- Introduzir Neural ODEs e o pacote `torchdiffeq`.
- Comparar autodiff padrão ($O(L)$) vs método adjunto ($O(1)$).
- Introduzir PINNs (Physics-Informed Neural Networks).
- Implementar uma PINN para a equação do calor em 1D.

Equações diferenciais e dinâmica

EDO vetorial

$$\frac{dh}{dt} = f(h, t), \quad h(0) = h_0.$$

- $h(t) \in \mathbb{R}^d$: estado do sistema.
- f : campo vetorial que define a dinâmica.
- O objetivo numérico: aproximar $h(t)$ em uma malha de tempos.

Sistema de Lotka–Volterra

Modelo presa–predador

$$\begin{aligned}\frac{dx}{dt} &= \alpha x - \beta xy, \\ \frac{dy}{dt} &= \delta xy - \gamma y.\end{aligned}$$

- $x(t)$: população de presas;
- $y(t)$: população de predadores;
- $\alpha, \beta, \delta, \gamma > 0$: parâmetros do modelo.

No laboratório: implementar Euler em NumPy para esse sistema.

Método de Euler (revisão)

Dada a EDO

$$\frac{dh}{dt} = f(h, t), \quad h(t_0) = h_0,$$

com passo Δt , o método de Euler é:

$$h_{n+1} = h_n + \Delta t f(h_n, t_n).$$

- Aproximação de primeira ordem (local).
- Simples de implementar, bom como *baseline*.

Exercício 1.1 – Euler em NumPy

No notebook, função:

```
numpy_euler_solver(func, h0, t_span)
```

Tarefas:

- 1 Calcular $f_val = func(h, t_curr)$.
- 2 Atualizar $h = h + dt * f_val$.
- 3 Armazenar o valor em uma lista para construir a trajetória.

Neural ODE

Definição

$$\frac{dh}{dt} = f_{\theta}(h(t), t),$$

onde f_{θ} é uma rede neural (MLP, CNN, etc.).

- A solução $h(t_1)$ é obtida por um *solver* de EDO.
- Treinamos θ por gradiente descendente, usando *autodiff* através do solver.

O pacote torchdiffeq

Assinatura principal

```
odeint(func, y0, t, method='dopri5')
```

- `func`: função `func(t, y)` em PyTorch.
- `y0`: estado inicial (tensor).
- `t`: tensor 1D com tempos de interesse.
- `method`: solver numérico (ex.: 'dopri5').

Saída:

- Tensor de forma `[len(t), batch_size, dim]`.
- Primeira dimensão: tempo; última: dimensão do estado.

Autodiff padrão vs método adjunto

Autodiff padrão

- Usa *backprop* pela computação inteira.
- Armazena estados intermediários.
- Custo em memória: $O(L)$, onde L é $\#$ de passos.

Em `torchdiffeq`:

- `odeint` $\Rightarrow$ autodiff padrão.
- `odeint_adjoint` $\Rightarrow$ método adjunto.

Método Adjunto

- Integra uma EDO adjunta para o gradiente.
- Não precisa guardar todos os estados.
- Custo em memória: $O(1)$ (em relação a L).

Exercícios 2.1 e 3.1 – Comparação prática

Na função `compare_autodiff`:

- Usar `odeint` para obter `H_FINAL_STD` e calcular `LOSS_STD`.
- Usar `odeint_adjoint` para `H_FINAL_ADJ` e `LOSS_ADJ`.
- Medir a norma dos gradientes em cada caso.

Discussão:

- Por que o método adjunto economiza memória?
- Como isso pode impactar o tempo de treino?

Physics-Informed Neural Networks (PINNs)

Ideia: usar a física (EDPs) como **regularizador** na loss.

- Rede aproxima solução: $u_{\theta}(x, t)$.
- Impomos:
 - Condições de contorno/iniciais;
 - Residual da EDP ≈ 0 no domínio.

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{boundary}} + \mathcal{L}_{\text{física}}.$$

Equação do calor 1D

Equação do calor

$$u_t(x, t) = \nu u_{xx}(x, t).$$

- $u(x, t)$: temperatura;
- ν : coeficiente de difusão.

Condição inicial usada no laboratório:

$$u(x, 0) = \sin(\pi x), \quad x \in [-1, 1].$$

Solução analítica:

$$u(x, t) = e^{-(\pi^2 \nu)t} \sin(\pi x).$$

PINN para a equação do calor

Rede neural $u_\theta(x, t)$ com entrada (x, t) :

- $\mathcal{L}_{\text{boundary}} = \frac{1}{N_b} \sum (u_\theta(x_i^b, t_i^b) - y_i^b)^2.$
- $\mathcal{L}_{\text{física}} = \frac{1}{N_f} \sum r_\theta(x_j, t_j)^2,$
com $r_\theta(x, t) = u_t - \nu u_{xx}.$

Em JAX:

- u_t via `grad` em relação a t .
- u_{xx} via `grad(grad(...))` em relação a x .

Exercício 4.2 – Resíduo da EDP

Na função `pde_residual(params, x, t, nu)`:

- 1 Definir $u_\theta(x, t)$ usando `mlp_forward`.
- 2 Calcular:

$$u_t = \frac{\partial u_\theta}{\partial t}, \quad u_{xx} = \frac{\partial^2 u_\theta}{\partial x^2}.$$

- 3 Retornar o resíduo:

$$r_\theta(x, t) = u_t - \nu u_{xx}.$$

Visualização da solução PINN

Após o treino:

- Gerar um **heatmap** de $u_\theta(x, t)$ no domínio (x, t) .
- Escolher um t^* e comparar:
 - Curva da PINN: $u_\theta(x, t^*)$;
 - Curva analítica: $e^{-(\pi^2\nu)t^*} \sin(\pi x)$.

Isso ajuda a:

- Diagnosticar se a PINN está aprendendo a física.
- Visualizar erros localmente em x e t .

Resumo da aula

- Vimos EDOs e o método de Euler no caso Lotka–Volterra.
- Introduzimos Neural ODEs com `torchdiffeq`.
- Comparámos autodiff padrão e método adjunto.
- Construámos uma PINN para a equação do calor em JAX.
- Discutimos como visualizar e interpretar as soluções.

Dúvidas?